Machine Intelligence

13. Reinforcement Learning

Learning From Experience

Álvaro Torralba

AALBORG UNIVERSITET

Fall 2024

In Previous Chapters of the MI Lecture...

Supervised Learning (Chapters 5-7)

- Learn a function from input/output examples
- Task: predict some (unobserved) target or class variable based on observed values of (predictor) attributes
  - Regression, if target is continuous
  - Classification, if target is discrete
- Model types e.g.: Decision trees, k-nearest neighbors, Neural networks, Naive Bayes,...

Unsupervised Learning (Chapter 8)

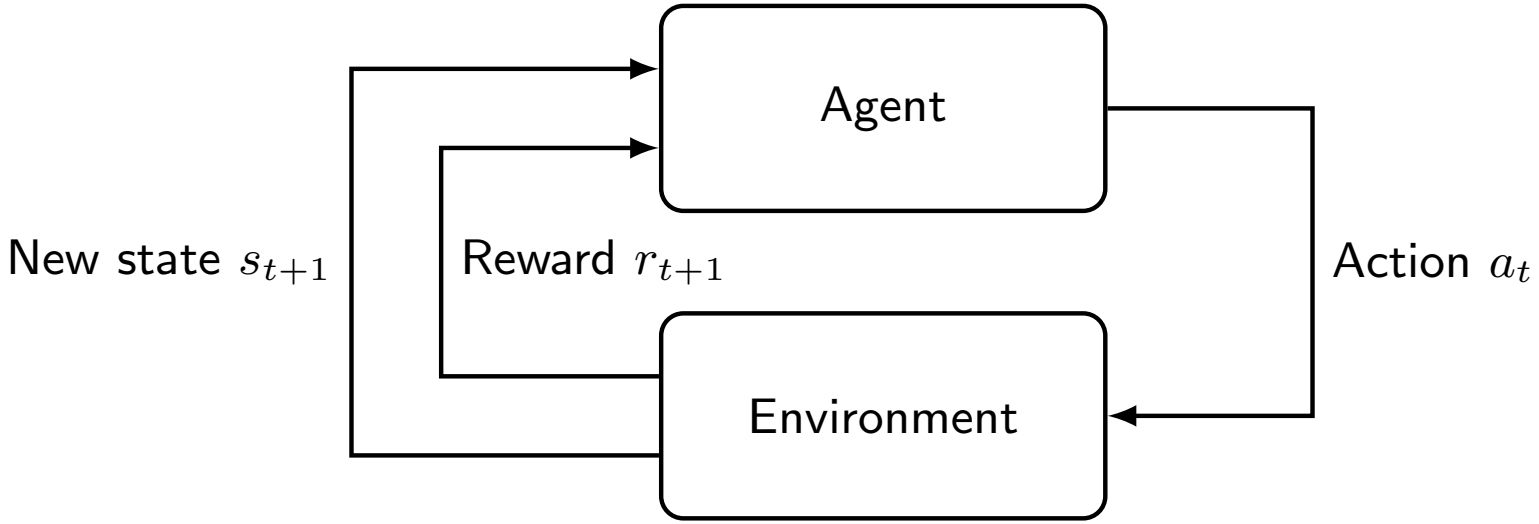
- Learn patterns in the input data
- Task: Clustering: identify coherent subgroups in data
- Model types e.g.: k-means, hierarchical clustering, Self-organizing maps, probabilistic clustering,...

Reinforcement learning (This Chapter)

- Task: Learn how to take actions in an environment aiming to maximize a cumulative reward

Reinforcement Learning

Reinforcement Learning: Learn when acting in an environment



- Agent interacts with an environment
- Every time it executes an action, the agent gets from the environment:
  - Observations: here, we assume it observes the full next state
  - **Reward** signal: that the agent wants to maximize
- The agent needs to learn in an **online** fashion, as its acting in the environment

Recap from Chapter 12

Definition (Markov Decision Process). *An* **MDP** *is a 6-tuple* $\Theta = (S, A, r, P, I, S^T)$ *where:*

- S *is a finite set of* **states**.
- A *is a finite set of* **actions**.
- $r : S \times A \times S \mapsto \mathbb{R}$ *is the* **reward function**.
- $P : S \times A \times S \mapsto [0, 1]$ *is the* **transition probability function**, *representing* $P(s' \mid s, a)$, *the probability that by applying action* a *in state* s, *we end in state* s'.
- $I \in S$ *is the* **initial state**.
- $S^T \subseteq S$ *is the set of* **terminal states**. *The set of terminal states could be empty.*

So, what is the difference?

Probabilistic Planning: Given a known MDP, how to compute the optimal policy.

Reinforcement Learning: What if transition probabilities (and rewards) are unknown a priori?

→ **We need to interact with the environment in order to learn!**

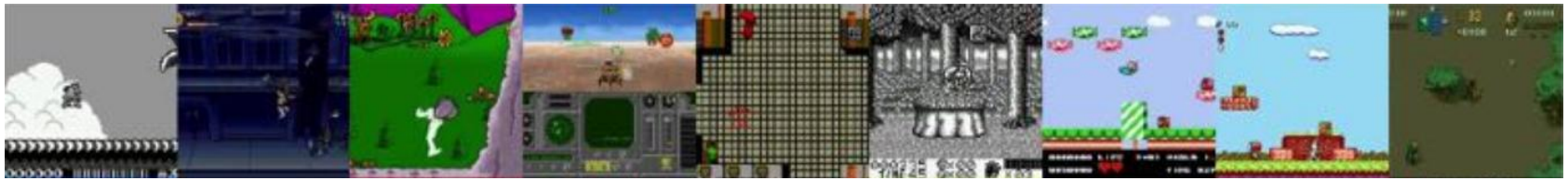
Model-free versus Model-based Reinforcement Learning

Question

So, what should we learn?

- Model-Free RL: Learn how to act. (learn a policy or value function)
- Model-based RL: Learn how the environment works (the transition probabilities and rewards), and act optimally according to that.

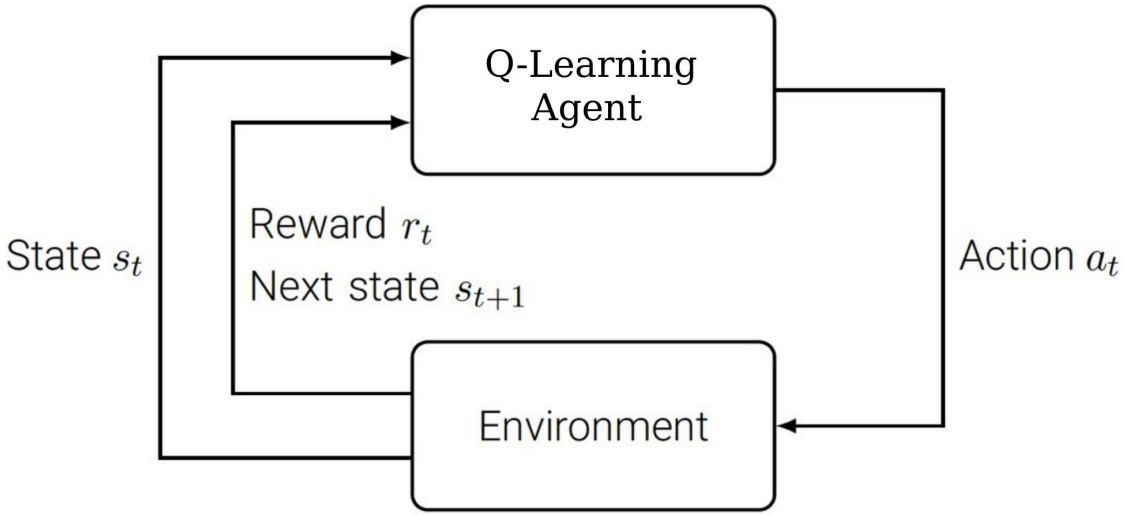
Q-Learning: Overview



http:blog.openai.com/gym-retro/

Initialize all $Q(s_t, a_t)$ with 0.
Repeat:

- Observe state s_t.
- **Choose** action a_t.
- Receive r_t and s_{t+1}.
- **Update** $Q(s_t, a_t)$.



Here: $\rightarrow$ What is $Q(s_t, a_t)$, and how to update it?

Next section: $\rightarrow$ How to choose actions?

Model-free Reinforcement Learning

We do not need to learn how the environment works. It may be enough to learn what to do in each state.

In the previous lecture, we considered:

- **Value function** $V^*(s)$: expected utility starting in s and afterwards acting optimally.

$$V^*(s) = R(s) + \max_{a \in A} \gamma \sum_{s'} P(s'|s,a) V^*(s')$$

Here, we consider instead:

- **Q-value** $Q^*(s,a)$: expected utility starting in s, applying action a, and afterwards acting optimally.

$$Q^*(s,a) = R(s) + \gamma \sum_{s'} P(s'|s,a) V^*(s')$$

We can still get V from Q: $V^*(s) = \max_a Q^*(s,a)$

So, what is the advantage of considering Q-values?

$\rightarrow$ We know what the best action is without knowing the transition relation: $\text{argmax}_a \, Q^*(s,a)$

Aside: Online Mean Estimation

Difficulty: Compared to supervised learning, we received experiences one by one. One option is to accumulate all previous experiences, construct a database and learn from such dataset
However:

- Is that how you learn?
- Is it really necessary to memorize all previous experiences?

Let's consider a different problem for a moment. We want to compute the average value ($\hat{x}$) of a list of numbers $x_1, x_2, x_3, \ldots$, but we receive numbers one by one.

- x_1: 10 $\rightarrow$ average = 10
- x_2: 20 $\rightarrow$ average = 15
- x_3: 9 $\rightarrow$ average = 13
  $\ldots \rightarrow$ average = 11
- x_{10}: 30 $\rightarrow$ average =

$$\hat{x}_{n+1} = \hat{x}_{n-1} + \frac{x_n - \hat{x}_{n-1}}{n} = (1 - \frac{1}{n})\hat{x}_{n-1} + \frac{1}{n}x_n$$

We update our belief based on each observation (and we do not need to explicitly store all the individual observations). $\rightarrow$ We can do the same in RL

Q-learning

- Learn an approximate model of $Q^*(s, a)$ directly.
- Update current estimate Q of Q^* after each experience $\langle s_t, a_t, r_t, s_{t+1} \rangle$

Q-learning update

After $\langle s, a, r, s' \rangle$ update $Q(s, a)$:

$$Q(s, a) \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$$

- We do not need to know the transition probabilities or the reward function, because we get s' and r directly by sampling the environment.
- α is the **learning rate**. It is a parameter in $[0, 1]$ that controls how much we weight our previous experiences and the new one.

Question!

Does this take into account the probabilities in the transition function?

Yes, indirectly, as outcomes with lower probability will be sampled less often when interacting with the environment.

Q-Learning: TD Updates

These updates are also called Temporal-Difference (TD) updates.

$$Q(s,a) + \alpha(\underbrace{\underbrace{R(s) + \gamma \max_{a'} Q(s',a')}_{\text{target}} - \underbrace{Q(s,a)}_{\text{prediction}}}_{\text{error (called temporal difference, TD)}})$$

- This formula is the same as the one in the previous slide, but refactored to highlight that what we add to the current value depends on the "error" of our prediction and the target.

Tabular Q-Learning Algorithm

Input: States S, Actions A, Discount factor γ, Learning rate α

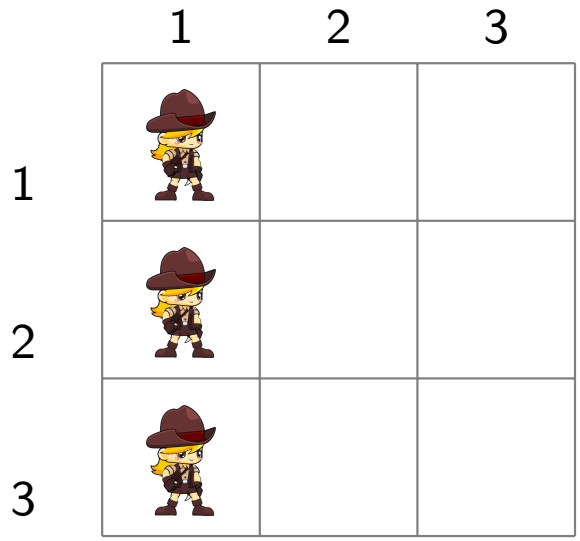
Initialize $Q(s, a)$ arbitrarily (e.g. initialize to 0 for all s, a);
$s \leftarrow$ Initial state;
repeat
    $a \leftarrow$ Select action ;
    $s', r \leftarrow$ execute a in s and observe state and reward ;
    expected_value $\leftarrow r + \gamma \cdot \max_{a' \in A} Q(s', a')$;
    $Q(s, a) \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha \cdot$ expected_value ;
    $s \leftarrow s'$;
    if s *is terminal* **then**
        $s \leftarrow$ start new episode (from initial state or some arbitrary state;
until *until convergence*;

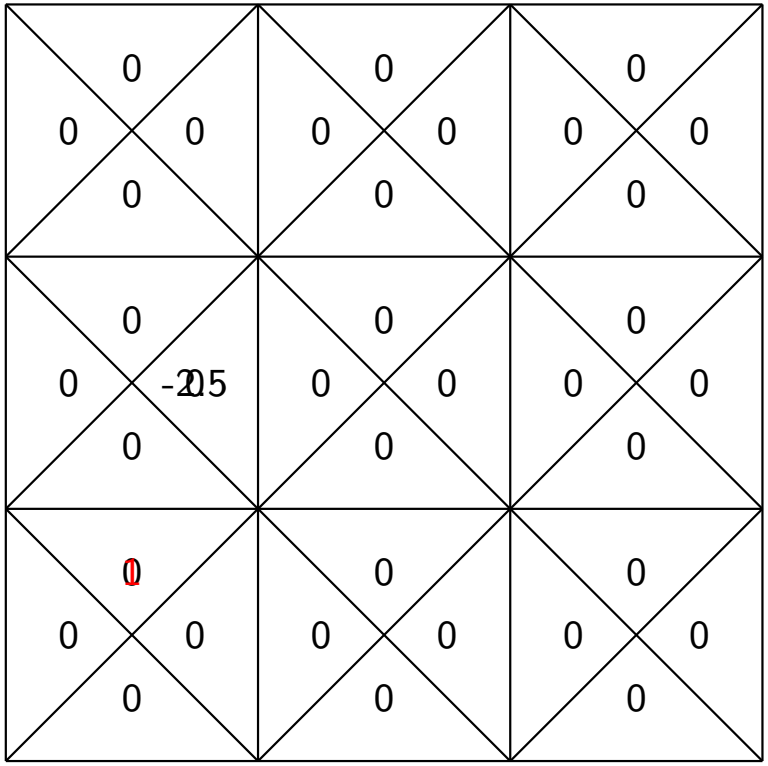
Algorithm 1: Q-learning

- As the agent interacts with the environment by executing actions, it learns the Q-value function
- This allows the agent to select better actions in the future (we disccuss how actions are selected in the next slides)

Q-learning: Example!

$\alpha = 0.5$, $\gamma = 0.9$





Pick an action: north, you get +2 of reward!

expected_value $= r + \gamma \cdot \max_{a' \in A} Q(s', a') = 2 + 0.9 \cdot 0 = 2$

$Q(s, a) = (1 - \alpha) \cdot Q(s, a) + \alpha \cdot$ expected_value $= 0 + 0.5 \cdot 2 = 1$

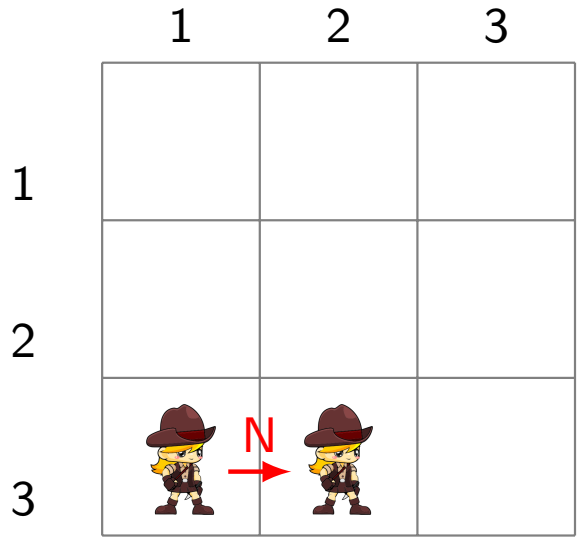
Pick an action: east, you get -5 reward!

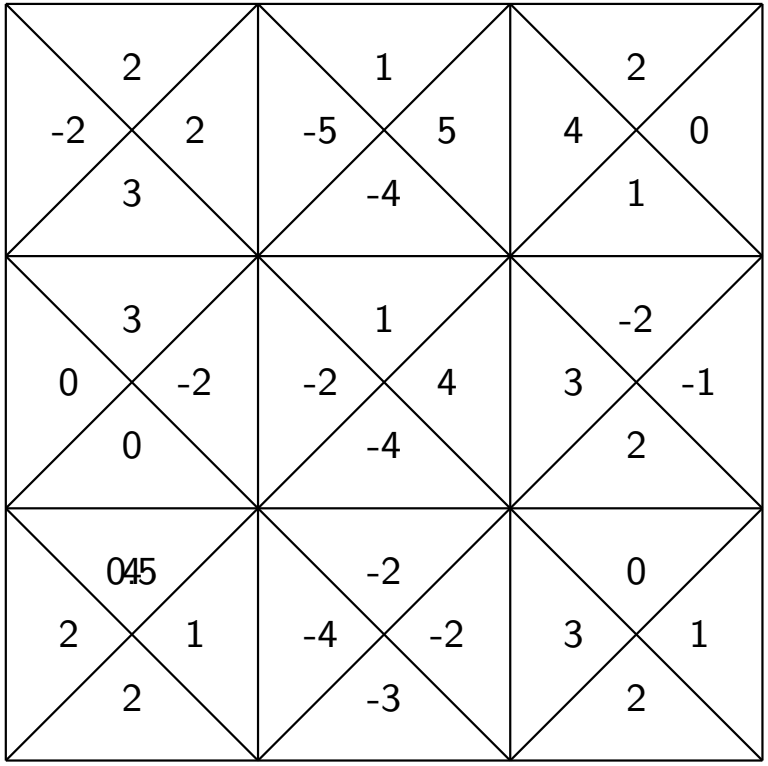
expected_value $= r + \gamma \cdot \max_{a' \in A} Q(s', a') = -5 + 0.9 \cdot 0 = -5$

$Q(s, a) = (1 - \alpha) \cdot Q(s, a) + \alpha \cdot$ expected_value $= 0 + 0.5 \cdot -5 = -2.5$

Q-learning: Example!

$\alpha = 0.5$, $\gamma = 0.9$



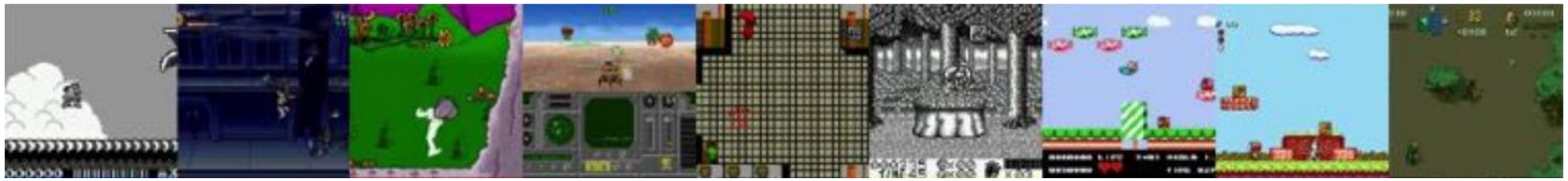


Question!

Pick an action: north, you get -1.2 of reward!
Which cell is updated, and to which value?

expected_value $= r + \gamma \cdot \max_{a' \in A} Q(s', a') = -1.2 + 0.9 \cdot -2 = -3$
$Q(s, a) = (1 - \alpha) \cdot Q(s, a) + \alpha \cdot$ expected_value $= 0.5 \cdot 4 + 0.5 \cdot -3 = 0.5$

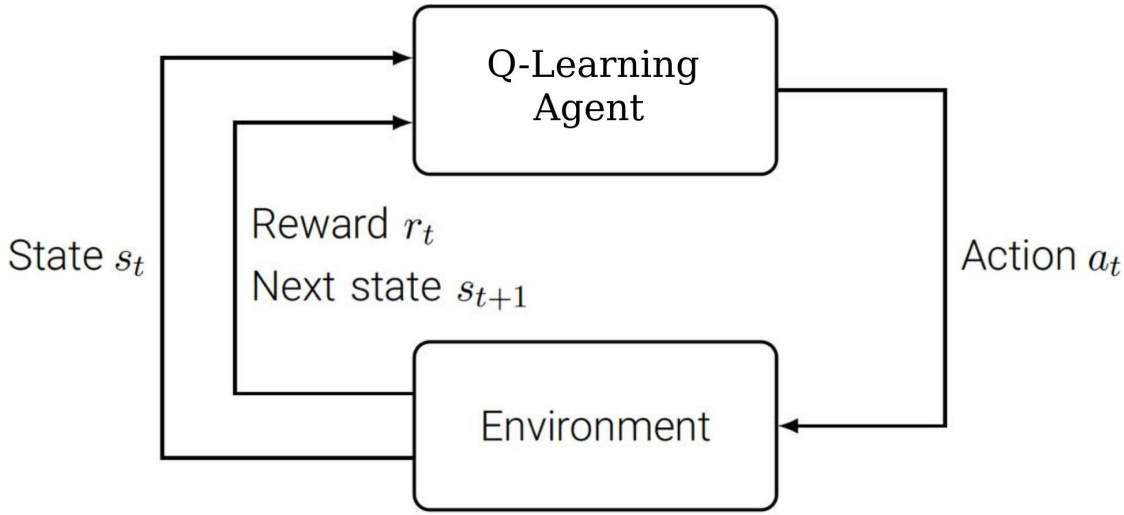
Q-Learning: Overview, Revisited



http:blog.openai.com/gym-retro/

Initialize all $Q(s_t, a_t)$ with 0.
Repeat:

- Observe state s_t.
- **Choose** action a_t.
- Receive r_t and s_{t+1}.
- **TD-update** $Q(s_t, a_t)$.



Previous section: $\rightarrow$ **What is $Q(s_t, a_t)$, and how to update it?**

Here: $\rightarrow$ **How to choose actions?**

Exploration vs. Exploitation

The famous trade-off. When choosing and executing action a, one has to balance two objectives:

- Exploration: Try new experiences.
  Intuition: Choose a for which the estimate $P(S' \mid s, a)$ is not yet accurate and/or the reward $R(s, a)$ is not yet known.

  $\rightarrow$ **Needed to converge to optimality.**

- Exploitation: Focus on things we already know work well.

  Choose a for which $R(s, a)$ is high, and/or $P(S' \mid s, a)$ gives high probability to states s' with high $U^c(s')$.
  $\rightarrow$ **Needed for effective training (otherwise may take forever), and to avoid excessive bad rewards if we're training in the real world (i.e., not in a simulator).**

- Example: Converging to optimal driving paths in a new city at rush hour through trial-and-error ...

$\rightarrow$ ϵ-greedy is the simplest method to address the exploration vs. exploitation trade-off in Q-learning.

The Greedy Policy, and ϵ-Greedy Action Selection

Definition (Greedy Policy). *Let $\Theta = (S, A, T, I, R, \gamma)$ be a discounted MDP, and $Q : S \times A \mapsto \mathbb{R}$. The* **greedy policy** *with respect to Q, denoted π^Q, is defined by $\pi^Q(s) = \arg\max_a Q(s, a)$.*

→ The greedy policy follows the current Q-value estimation.

ϵ-greedy strategy: For some $0 < \epsilon < 1$.

- With probability ϵ: Pick an action randomly
- With probability $(1 - \epsilon)$: Pick the action of the greedy policy

Typically the value of ϵ is close to 0, but there is also the possibility of starting with higher values and decreasing ϵ as the agent gains experience

→ The ϵ-greedy policy *exploits* with probability $1 - \epsilon$ (greedy action), and *explores* with probability ϵ (random applicable action).

Q-Learning Properties

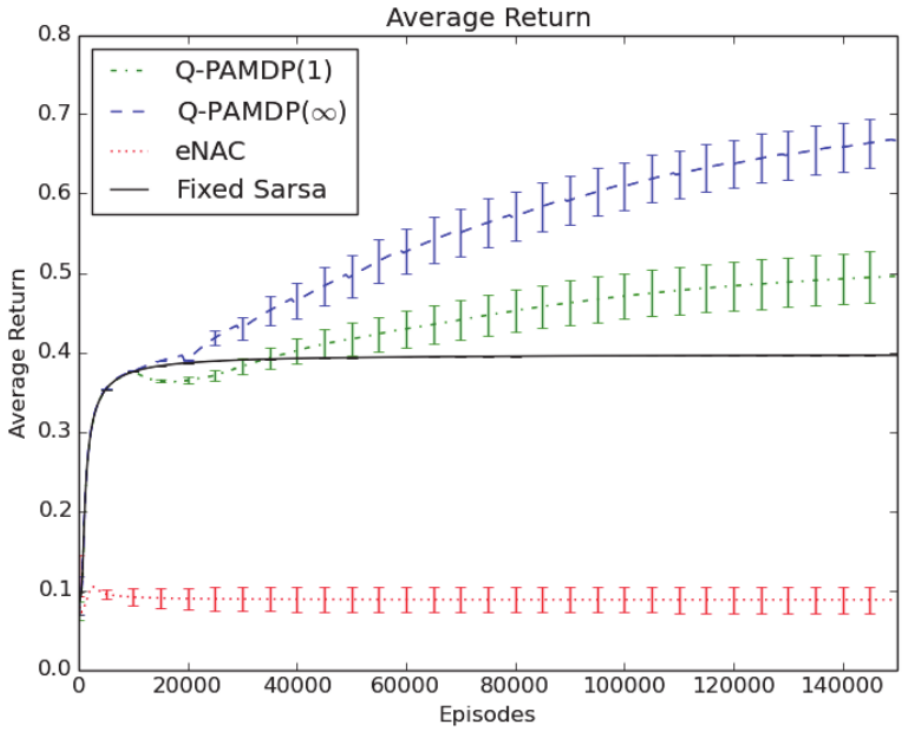
Q-Learning is **guaranteed to coverge to the optimal value if** the following properties hold:

1. **All states and actions are visited infinitely often**
   $\rightarrow$ For convergence you don't need to be acting optimally (or even close enough), but you must do enough exploration
2. **Adjust the learning rate** α such that $\sum_{t=0}^{\infty} \alpha_t = \infty$ and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$

So, α needs to be decreased over time (at some point your experience is more important than any new information you obtain from the environment) but without reaching 0 (never stop learning!)

Example:
$1 + \frac{1}{2} + \frac{1}{3} \cdots = \infty$
$1 + \frac{1}{2^2} + \frac{1}{3^2} \cdots = \frac{\pi^2}{6} < \infty$

Evaluation of RL Agents



- How much reward they accumulate over time.
  $\rightarrow$They should perform well as they learn (so this evaluates both how much the agent learns as well as the exploitation/exploration strategy)
- After training for a fixed amount of time, how good is the resulting policy
  $\rightarrow$ While evaluating the policy, the agent only selects actions greedily (is not focused on exploration anymore)

Real World versus Simulator

Learning in the real world is difficult (and may be dangerous)

- When executing actions, the agent could cause harm!
  $\rightarrow$ Are you sure you want to do random exploration in real life?
- Gaining real experience is very expensive
  $\rightarrow$ For example, if it requires interacting with users

Potential solution: use a **simulated environment**

Drawback: If you don't know the transition probabilities or the reward function, how do you construct such a simulator?

$\rightarrow$ **If you have the model of the environment, you should also consider (probabilistic) planning algorithms**

The Problem with Q-Learning

Q-function array: size $|S| \cdot |A|$, way too large on most interesting problems

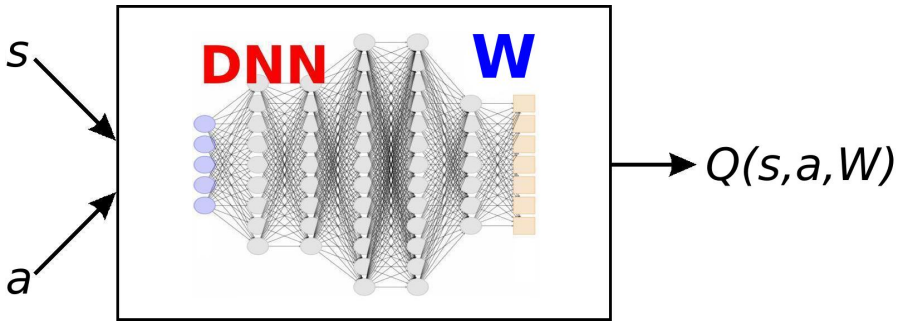
$\rightarrow$ To the rescue: neural networks as function approximator for $Q(s_t, a_t)$.

From Q-Learning to Deep Q-Learning

Idea: Replace array $Q[s, a]$ with (deep) neural network (DNN).

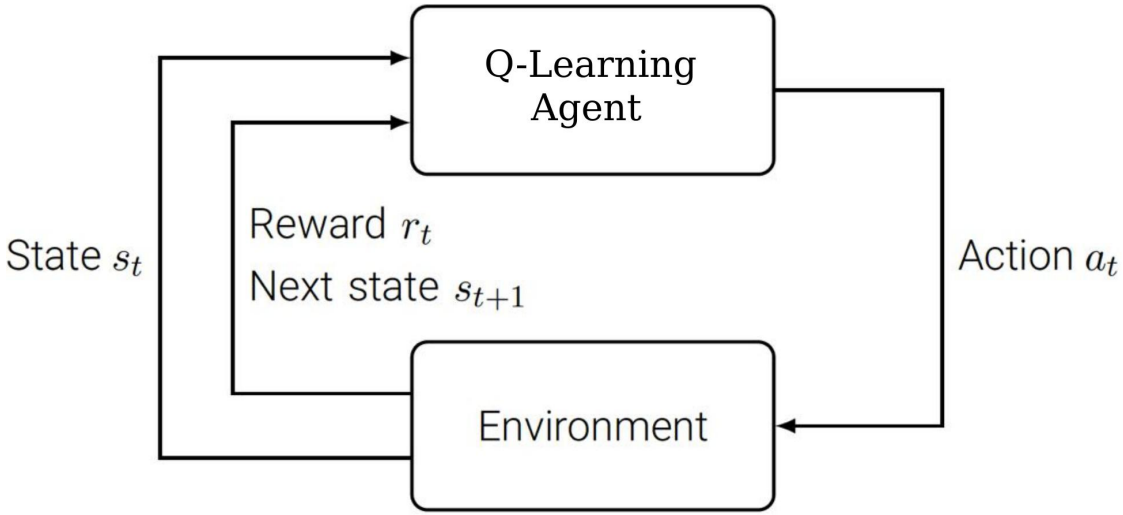
- Function approximator to estimate Q.
- Weights W, $\mathbf{Q(s, a, W)} \approx Q^*(s, a)$.
- **Generalizes over** s, a instead of memorizing every individual pair.

$\rightarrow$ From arrays to machine learning.



Initialize weights W.
Repeat:

- Observe s_t.
- Choose a_t: same as before.
- Receive r_t and s_{t+1}.
- Update W: **Learning step**.



Deep Q-Learning: The Learning Step

Loss function:

$$L(s_t, a_t, W) \leftarrow$$
$$[R(s_t) + \gamma \max_{a_{t+1} \in A(s_{t+1})} Q(s_{t+1}, a_{t+1}, W) - Q(s_t, a_t, W)]^2$$

$\rightarrow$ Squared error in Q-values, i.e., TD error squared.

Backpropagation:

$$W \leftarrow W - \alpha \nabla_W L(s_t, a_t, W)$$

$\rightarrow$ Stochastic gradient descent step with respect to the weights W.

Deep Q-Learning: Experience Replay

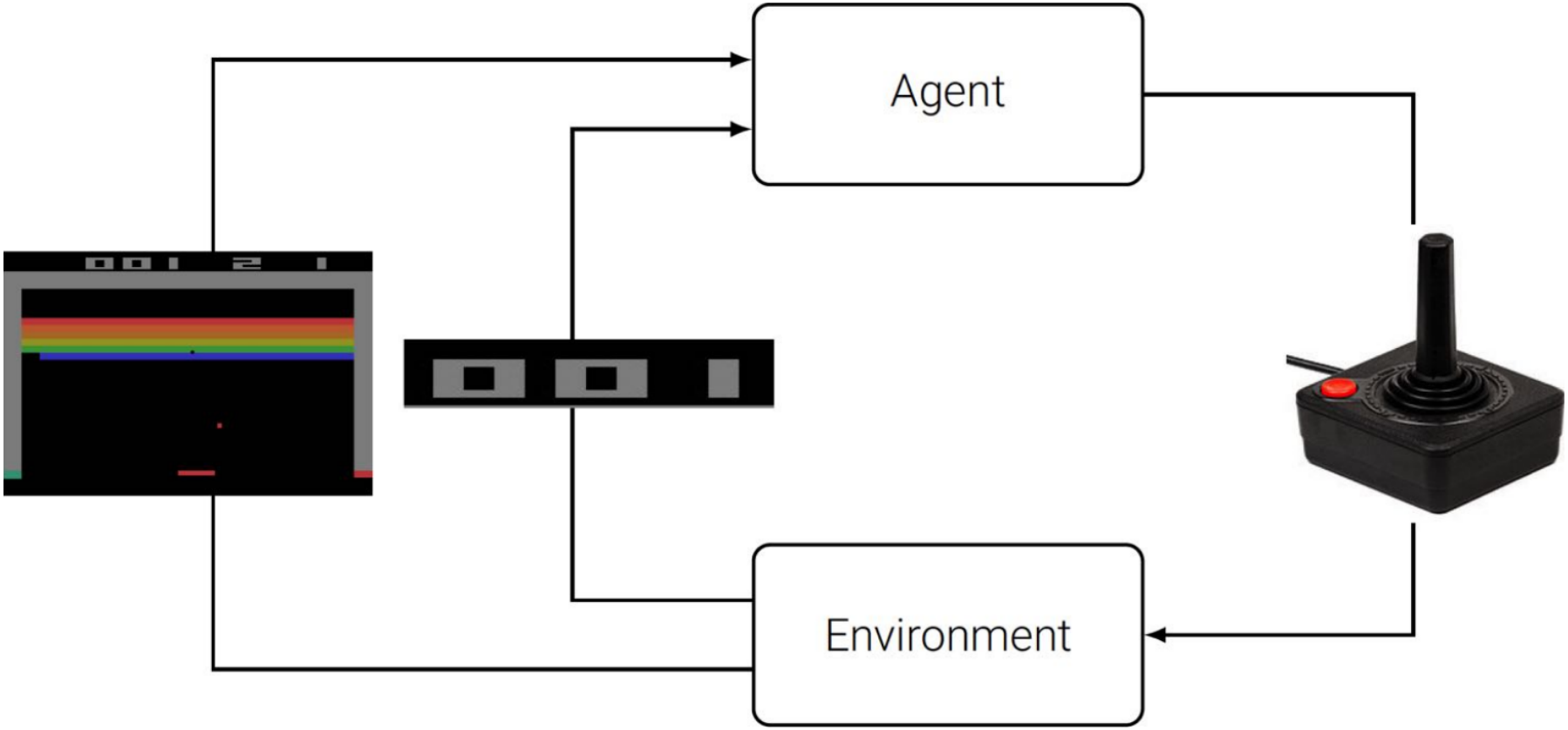
Consecutive samples: Are problematic for ML.

- Tabular ϵ-Greedy Q-learning as per slide **??** trains from consecutive samples.
- Problem: strong correlations between consecutive samples.
- ML on correlated samples is inefficient.

Experience replay: Train in mini-batches instead.

- Maintain a **replay buffer** B storing samples (s_t, a_t, r_t, s_{t+1}).
- Train on samples (s_t, a_t, r_t, s_{t+1}) drawn randomly from B.
- Breaks correlation between samples.
- Improves data efficiency as each sample can be used multiple times.

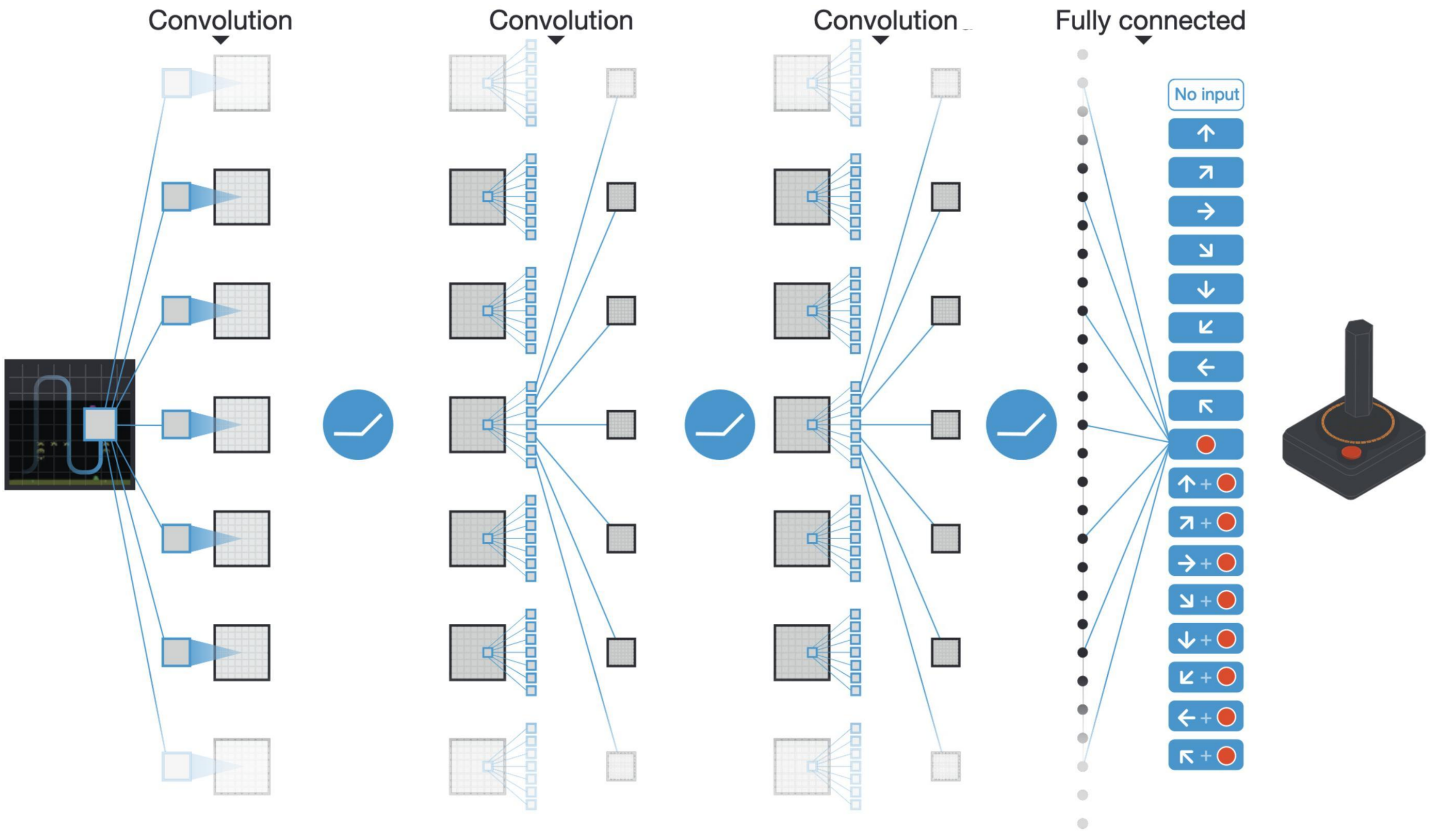
Case Study: Atari Games [?]



Objective: maximize game score

Neural Network Architecture

Representation of $Q(s, a, W)$:

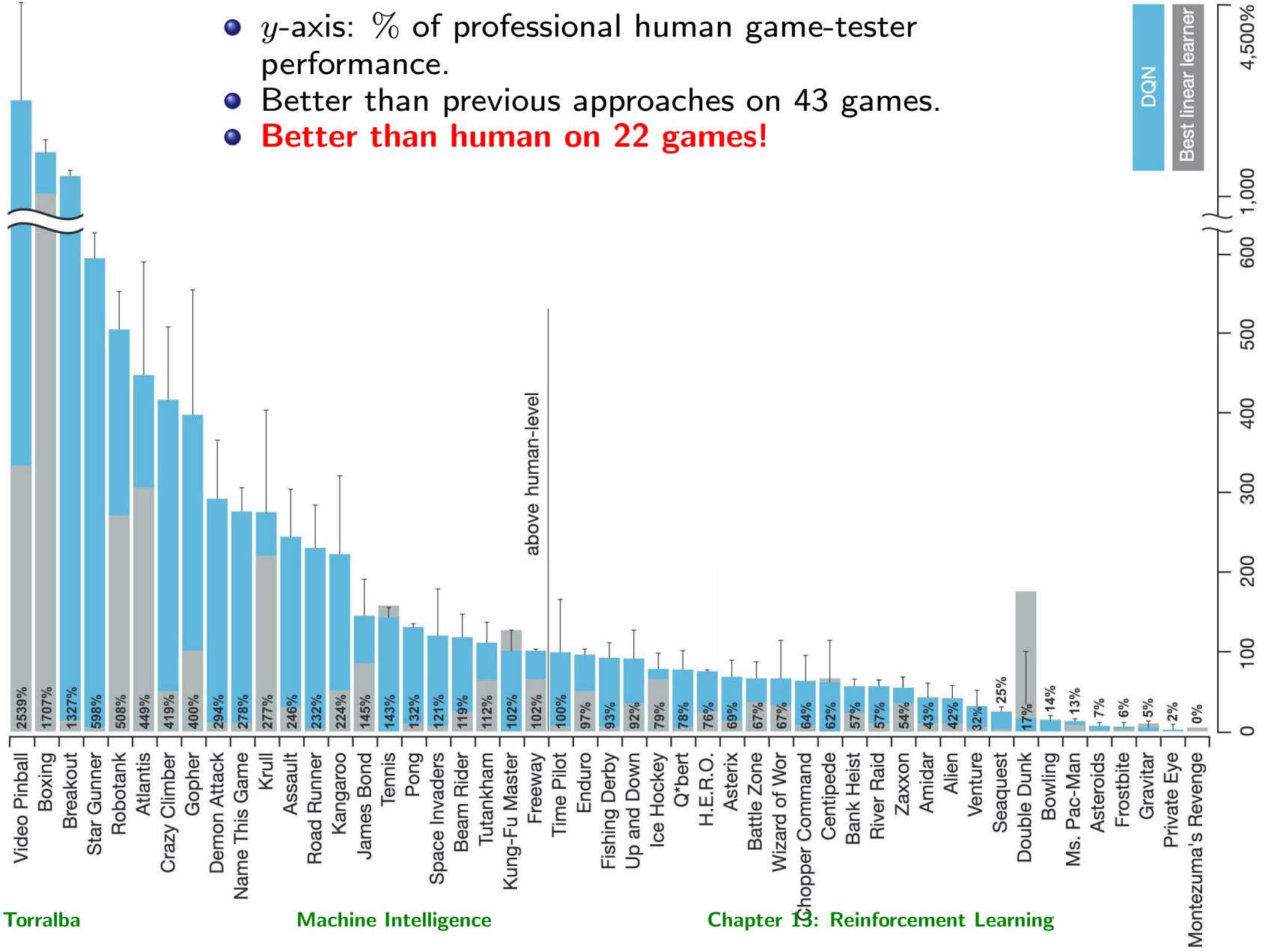


Input: $84 \times 84 \times 4$ (stack of last 4 frames). **Output:** Q-values.

Architecture: 32 8×8 filters $\rightarrow$ ReLU $\rightarrow$ 64 4×4 filters $\rightarrow$ ReLU $\rightarrow$ 64 3×3 filters $\rightarrow$ ReLU $\rightarrow$ 256 neurons $\rightarrow$ ReLU $\rightarrow$ 18 neurons.

Results

- y-axis: % of professional human game-tester performance.
- Better than previous approaches on 43 games.
- **Better than human on 22 games!**



Model-based Reinforcement Learning

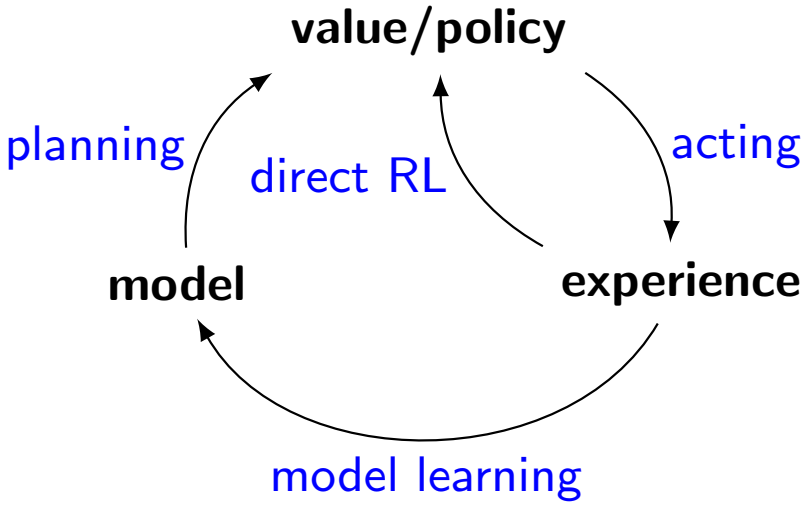
Can learn transition probabilities from example traces:

$$\begin{aligned} t_1: &\quad s_{0,1}a_{0,1}s_{1,1}a_{1,1}s_{2,1}a_{2,1}s_{3,1}\ldots \\ t_2: &\quad s_{0,2}a_{0,2}s_{1,2}a_{1,2}s_{2,2}a_{2,2}s_{3,2}\ldots \\ &\ldots \end{aligned}$$

- Known are state and action sets S, A.
- Maintain current model P^c, U^c for transition probabilities and value function
- Do:
  - observe current state s
  - choose and execute action a
  - observe new state s' and reward $R(s,a)$
  - update estimate for $P(s' \mid s,a)$ and $U(s)$

At every step, we can apply planning in order to act optimally according to our current belief over the transition probabilities and value function!

Summary: MDPs and RL



Known MDP: Offline solution

Goal	Technique
Compute V^*, Q^*, π^*	Value/policy iteration
Evaluate a fixed policy π	Policy evaluation

Unknown MDP: Model-based

Goal	Technique
Compute V^*, Q^*, π^*	VI/PI on approx. MDP
Evaluate a fixed policy π	PE on approx. MDP

Unknown MDP: Model-free

Goal	Technique
Compute V^*, Q^*, π^*	Q-Learning
Evaluate a fixed policy π	TD-Learning

Conclusions

- **Reinforcement Learning** is how an agent learns how to act by acting in an environment and receiving a certain **reward** signal.
- **Q-learning** is an RL algorithm that learns the Q-value function.
- Q-learning always converges to the optimal value function if there is enough exploration, and the learning rate is adjusted in the right way.

Further Material

- If you are interested in the topic, you can watch the 6-lecture videos of Pieter Abbeel on Reinforcement Learning, where they cover parts of what we saw in Chapters 12 and 13: `https://www.youtube.com/watch?v=2GwBez0D20A`

The exam

- January 13th, 2023, 12:00 to 16:00.
- Written exam in Moodle with internal censors.
- Answers should be written in English.
- We will use Moodle and the IT-Flex Platform. Software that you can use:
  - PDF viewers (in particular, you can download and check during the exam all material from the lecture)
    $\rightarrow$ However, **don't rely on doing this**, the exam requires time and if you have to check for most of the questions, there won't be enough time for everything
  - Calculator
- Software that you cannot use:
  - Software like Weka, or Hugin is not needed
  - Excel or other programming tools
  - Google (basically navigating the web is not allowed)

What the course has covered

The course has covered the following issues:

- Introduction
- Constrained satisfaction problems
- Reasoning under uncertainty
- Bayesian networks
- Supervised Learning
- Unsupervised Learning
- Search-based methods
- Planning: Classical, Multi-agent and probabilistic
- Reinforcement Learning

This corresponds to the following literature:

- The slides from the course.
  $\rightarrow$ **Main study material. All 13 chapters/lectures are relevant for the exam. Parts that are not relevant are marked as "Further material" or "(for reference)". All the rest is relevant for the exam.**
- David L. Poole and Alan K. Mackworth, Artificial Intelligence: Foundations of computational agents (Third edition): (sections relevant for each of the lectures available in Moodle).

Exam Questions

- You may expect written exercises of similar nature to the ones you solved in the exercise sessions.
  $\rightarrow$Of "similar nature" does not mean that the exercises will be like a template, you need to understand the concepts and be able to apply it in different ways
- You can find previous exams on Moodle. This can be used as additional exercises. However, note that they are from previous editions of the course, so don't assume that the new exam will look exactly like those.